📚 GUIA DE ESTUDOS – SISTEMA OPERACIONAL: SOFTWARES

Tema 03

A Alma da Máquina: Entendendo os Softwares

---

1. O que é Software?

💡 **Conceito-chave (bem direto):**
Software é a parte lógica do computador - o conjunto de instruções que diz ao hardware o que fazer. É o que "dá vida" às máquinas.

🧠 **Em poucas palavras:**
Se o hardware é o corpo do computador, o software é a mente. Sem software, o computador é apenas um conjunto de componentes eletrônicos sem função.

📌 **Importante para iniciantes:**

•Software é **intangível** (não podemos tocar)
•É composto por **códigos, funções, bibliotecas e módulos**
•Funciona dentro da estrutura física do hardware
•Segue o ciclo: **Entrada (Input)** → **Processamento** → **Saída (Output)**

Onde os softwares ficam armazenados?

•Permanentemente: em dispositivos **não voláteis** (HD, SSD, memória flash)
•Durante a execução: são carregados na **memória RAM** (volátil)

---

2. Classificação dos Softwares

Os softwares podem ser classificados de diversas formas. Vamos explorar cada uma delas:

2.1 Classificação pela Licença

Tipo	Descrição	Exemplos	Características
Proprietário	Criado e distribuído por empresas com licenças limitadas	Microsoft Office, Adobe Photoshop, Windows	Código-fonte fechado; pagamento por licença
Livre / Open Source	Desenvolvido colaborativamente; código-fonte acessível	Linux, Mozilla Firefox, LibreOffice, Android	Pode ser modificado; nem sempre é gratuito

⚠ **Importante:** Software livre NÃO é sinônimo de software gratuito! A "liberdade" se refere ao acesso ao código, não necessariamente ao preço.

2.2 As Quatro Liberdades do Software Livre (FSF)

A Free Software Foundation define quatro liberdades essenciais:

Liberdade	Descrição	O que permite
0	Executar	Usar o programa para qualquer finalidade
1	Estudar	Acessar e entender o código-fonte
2	Distribuir	Compartilhar cópias com outras pessoas
3	Melhorar	Modificar e publicar as melhorias

2.3 Classificação pela Finalidade

Categoria	Função	Exemplos
Sistemas Operacionais	Gerenciam hardware e fornecem interface	Windows, Linux, macOS, Android, iOS
Produtividade	Aumentam eficiência em tarefas	Word, Excel, PowerPoint, Google Docs

Categoria	Função	Exemplos
Design Gráfico	Criam e editam imagens/ilustrações	Photoshop, Illustrator, GIMP, CorelDRAW
Edição de Vídeo	Manipulam e editam vídeos	Premiere Pro, Final Cut, DaVinci Resolve
Banco de Dados	Armazenam e gerenciam dados	MySQL, Oracle, SQL Server, PostgreSQL
Navegação Web	Acessam a internet	Chrome, Firefox, Edge, Safari
Segurança	Protegem contra ameaças	Avast, Norton, McAfee, Kaspersky

2.4 Classificação pela Plataforma

Plataforma	Descrição	Exemplos
Desktop	Para computadores pessoais	Windows, Photoshop, Word
Web	Executados em servidores, acessados via navegador	Gmail, Google Docs, Trello
Mobile	Para smartphones e tablets	Apps Android/iOS, WhatsApp, Instagram
Embarcados	Integrados a dispositivos específicos	Sistemas de carros, eletrodomésticos, equipamentos médicos

2.5 Classificação pelo Modo de Execução

Modo	Descrição	Exemplos
Nativo	Compilado para arquitetura específica	Programas em C/C++ compilados
Interpretado	Executado por um interpretador, sem compilação prévia	Python, JavaScript, PHP
Virtual	Executado em máquina virtual	Java (JVM), C# (.NET)

3. Tipos de Software (Detalhamento)

3.1 Sistemas Operacionais

O sistema operacional é o software mais importante do computador. Ele:

•Gerencia os recursos do hardware (memória, processador, dispositivos)
•Fornece interface para o usuário interagir com a máquina
•Permite a execução de outros programas (aplicativos)

Exemplos: Windows, Linux, macOS, Android, iOS

3.2 Softwares Aplicativos

São programas desenvolvidos para realizar tarefas específicas para o usuário.

🔹 Aplicativos de Produtividade

•**Editores de texto:** Word, Google Docs, LibreOffice Writer
•**Planilhas eletrônicas:** Excel, Google Sheets, LibreOffice Calc
•**Apresentações:** PowerPoint, Google Slides, LibreOffice Impress
•**Gerenciamento de projetos:** Trello, Asana, Monday.com

🔹 Softwares de Design Gráfico

•**Edição de imagens:** Adobe Photoshop, GIMP, CorelDRAW
•**Ilustração vetorial:** Adobe Illustrator, Inkscape
•**Diagramação:** Adobe InDesign, Canva

🔹 Softwares de Edição de Vídeo

•**Profissionais:** Adobe Premiere Pro, Final Cut Pro, DaVinci Resolve
•**Semi-profissionais:** Sony Vegas, Filmora
•**Gratuitos:** Shotcut, OpenShot, iMovie

🔹 Softwares de Banco de Dados

•**Relacionais:** MySQL, PostgreSQL, Oracle, SQL Server

•**Não-relacionais (NoSQL):** MongoDB, Cassandra, Redis

- **Softwares de Navegação Web**

•**Navegadores:** Google Chrome, Mozilla Firefox, Microsoft Edge, Safari, Opera

- **Softwares de Segurança**

•**Antivírus:** Avast, Norton, McAfee, Kaspersky
•**Firewall:** Windows Defender Firewall, pfSense
•**Criptografia:** VeraCrypt, BitLocker

---

4. Desenvolvimento de Software

O desenvolvimento de software segue um **ciclo de vida**, com etapas bem definidas:

4.1 Etapas do Desenvolvimento

Etapa	Descrição	Atividades
1. Análise de Requisitos	Entender o que o software precisa fazer	Coletar dados com usuários, definir escopo, documentar necessidades
2. Projeto	Definir arquitetura e design	Criar diagramas, modelar banco de dados, projetar interface (UI/UX)
3. Codificação	Escrever o código-fonte	Programar em linguagem escolhida, seguir boas práticas
4. Testes	Verificar qualidade e corretude	Testes unitários, integração, sistema, aceitação; testes beta
5. Implantação	Disponibilizar aos usuários	Empacotar, distribuir, instalar, configurar
6. Manutenção	Corrigir e melhorar após o lançamento	Correção de bugs, patches de segurança, novas funcionalidades
7. Aposentadoria	Retirar de uso	Migrar dados, encerrar suporte

---

4.2 Tipos de Testes

Tipo	Descrição
Teste Unitário	Testa componentes individuais (funções, métodos)
Teste de Integração	Verifica se os componentes funcionam juntos
Teste de Sistema	Testa o sistema completo
Teste de Aceitação	Validado pelo cliente/usuário final
Teste Beta	Versão para um grupo limitado de usuários testarem e darem feedback

---

5. Linguagens de Programação

Linguagens de programação são conjuntos de regras e símbolos que permitem aos programadores criar instruções que o computador pode entender e executar.

5.1 Evolução das Linguagens

Década	Linguagens	Características
1950	Fortran	Primeira linguagem de alto nível; científica/engenharia
1960	COBOL, PL/I	Aplicações comerciais e governamentais
1970	C	Criação de sistemas operacionais (Unix), baixo nível
1980	C++, Objective-C	Programação orientada a objetos
1990	Java, Python	Portabilidade (Java), simplicidade (Python)
2000+	C#, JavaScript, Ruby, Swift, Go, Kotlin	Web, mobile, cloud, ciência de dados

---

5.2 Principais Linguagens e suas Aplicações

Linguagem	Principais Aplicações
Python	Ciência de dados, IA, web, automação
Java	Aplicações empresariais, Android
JavaScript	Desenvolvimento web (front-end e back-end)
C/C++	Sistemas operacionais, jogos, embarcados
C#	Aplicações Windows, jogos (Unity)
PHP	Desenvolvimento web (back-end)
Swift	Aplicações iOS/macOS
Kotlin	Aplicações Android
Go	Sistemas distribuídos, cloud

---

6. Distribuição e Instalação de Softwares

6.1 Processo de Distribuição

1.**Preparação:** Reunir todos os arquivos necessários (executáveis, bibliotecas, documentação)
2.**Empacotamento:** Criar um arquivo compactado (.exe, .msi, .zip, .dmg)
3.**Distribuição:** Disponibilizar em lojas, sites, mídias físicas

Formatos comuns:

•Windows: .exe, .msi
•macOS: .dmg, .pkg
•Linux: .deb, .rpm
•Móveis: .apk (Android), .ipa (iOS)

6.2 Processo de Instalação

1.**Extrair** arquivos do pacote
2.**Configurar** opções e preferências

3.**Criar** atalhos e entradas no sistema
4.**Registrar** componentes no sistema operacional

6.3 Atualizações

Tipo	Descrição
Automática	Software verifica e instala atualizações sem intervenção
Manual	Usuário precisa baixar e instalar
Patch	Correção específica (geralmente segurança)
Service Pack	Conjunto de atualizações acumuladas

---

7. Licenciamento de Software

7.1 Tipos de Licença

Tipo	Descrição	Exemplos
Proprietária	Direitos autorais reservados; restrições de uso	Windows, Photoshop, Office
Freeware	Gratuito, mas código-fonte fechado	Skype, Adobe Reader
Shareware	Gratuito por tempo limitado ou com recursos limitados	WinRAR, muitos jogos
Open Source	Código-fonte aberto; licenças específicas	GPL, MIT, Apache
Trial	Versão de teste por tempo limitado	Muitos softwares comerciais
Demo	Versão com funcionalidades limitadas	Jogos, softwares profissionais

---

7.2 Principais Licenças Open Source

Licença	Características
GPL (General Public License)	Se você distribuir modificações, deve disponibilizar o código
MIT	Permissiva; permite uso em software proprietário

Licença	Características
Apache	Similar à MIT, com proteção de patentes
BSD	Permissiva; poucas restrições

---

8. Resumo Visual

```text
┌──────────────────────────────────────────────────────────────────┐
│                              SOFTWARE                            │
│   (Parte lógica do computador - "a mente")                       │
└──────────────────────────────────────────────────────────────────┘
                                  │
          ┌───────────────────────┼────────────────────────┐
          ▼                       ▼                        ▼

┌──────────────────┐     ┌──────────────────┐     ┌──────────────────┐
│SISTEMAS          │     │  APLICATIVOS     │     │ LINGUAGENS DE    │
│OPERACIONAIS      │     │                  │     │ PROGRAMAÇÃO      │
├──────────────────┤     ├──────────────────┤     ├──────────────────┤
│Windows           │     │Produtividade     │     │Python, Java      │
│Linux             │     │Design            │     │JavaScript, C++   │
│macOS             │     │Edição de Vídeo   │     │PHP, Swift        │
│Android           │     │Banco de Dados    │     │Go, Kotlin        │
│iOS               │     │Navegação         │     │Ruby, C#          │
│                  │     │Segurança         │     │                  │
└──────────────────┘     └──────────────────┘     └──────────────────┘
```

---

9. Perguntas para Reflexão

1.Qual a diferença entre software proprietário e software livre?
2.Quais são as quatro liberdades do software livre?
3.Por que a memória RAM é importante para a execução de softwares?
4.Qual a diferença entre software interpretado e software compilado?
5.Por que as atualizações de software são importantes?
6.Como escolher a linguagem de programação adequada para um projeto?